BulsuScholar
Security Questions and Answers

Thesis Reference: Implemented Controls, Provider Services and Remaining Gaps

Checked on: October 20, 2026 (Asia/Taipei). This report answers the requested questions using the current local source, read-only database metadata, public HTTPS checks, focused automated tests and official Supabase documentation.

Important boundary: This is a factual implementation review, not a penetration test, certification or guarantee that every endpoint is secure. Local source may differ from the deployed backend. No student documents, passwords, tokens or secret values were included in this report; no production settings were changed.

How to Read the Labels

Label	Meaning
Implemented	A control exists in the application code. Live confirmation is stated separately where available.
Provider-managed	Supabase or the hosting provider supplies the feature; this is not custom security code written by the project.
Not implemented / Future improvement	No complete project implementation was found for the stated control.
Not verified	The setting or operational evidence was not available or not checked. This does not mean the feature is definitely disabled.

Confirmed Live, Without Reading Student Records

The connected BULSUScholar database has **64 public tables, all with RLS enabled**. No public-table grants to PUBLIC, anon or authenticated were returned. The **bulsuscholar bucket is private**, with a 10 MB bucket limit and no storage-object policies returned. The configured login-attempt limit is **3**. An aggregate check found no password, encryptedPassword, passwordHash or temporaryPassword keys in the student, pending-student, provider and admin account JSON rows.

Public checks found HTTP-to-HTTPS redirects for both the website and API. A focused local run passed **94 backend tests**; those tests do not establish that every production workflow, backup or recovery operation has been tested.

1. Authentication: Login, Passwords and Verification

Implemented

What handles authentication?
The application uses **Supabase Auth together with custom FastAPI checks**. Supabase checks the password and issues login tokens. FastAPI checks the account type, account status, lockout state, identity and additional portal-session requirements. It is not a separate homemade password database.

Does the application store passwords?
The normal current account flow sends passwords to Supabase Auth, not to a password field in student or grantor records. The backend temporarily receives passwords during login or administrator-created account setup. The live aggregate check found zero listed password fields in the four account tables checked. Recovery records do keep a snapshot of the provider's password hash to check that a reset changed it; this is not a readable password or a separate login database. A legacy grantor-password migration script also exists; it is not the current password-storage design.

Must students confirm email before activation?
Yes. The confirmation workflow checks the authenticated user's confirmed email and matching student identity. With Manual Account Review enabled, the account remains pending until full-admin approval. In automatic mode it activates only when the required signup documents are ready. Grantor/admin accounts created by authorized staff may be marked email-confirmed through the administrative Auth API, so this is not the same email-link process for every role.

Is there failed-login lockout?
Yes, for the normal student, grantor and admin login route. The current database limit is 3 failed attempts; a full admin may set it from 3 to 10. Failures increase a database counter. Reaching the limit locks the account and revokes Auth sessions. Backend checks also reject remaining tokens for a locked account. A successful login resets the counter only if the account is not already locked. There is no normal fixed-time automatic unlock: students and grantors use verified recovery, while locked admins can be unblocked by Root.

Provider-managed

Supabase Auth stores passwords as salted **bcrypt hashes**, not readable passwords. Hashing is different from reversible encryption. Provider password rules and Auth settings are managed in Supabase; do not claim that optional leaked-password protection is enabled without checking its dashboard setting. [W1]

Not Implemented / Not Verified

The separate Root password-login route does not call the normal portal failed-login counter. Its additional code has a separate attempt limit, described on the next page. Do not claim identical lockout behavior across both login routes.

Evidence: backend/auth_service.py:136, 205-289, 419; backend/student_lifecycle_service.py:34; backend/signup_service.py:341; src/pages/SignupPage.jsx:1610; [D1, D2]

1. Authentication: OTP, Sessions and Account Recovery

Implemented

Is MFA or OTP used?
A conditional email-code step is implemented. Students and grantors need a six-digit email code after at least 30 days without recorded meaningful activity, unless a recovery grace period applies. It is not required at every login and is not applied to ordinary administrators by this rule. Codes expire in 10 minutes; the checks allow 3 wrong attempts, a 60-second resend delay and at most 5 issued challenges in an hour. Code hashes use HMAC-SHA256, with a server-only secret. This is a custom email verification step, not proof of Supabase authenticator-app MFA enrollment.

What extra check does Root Admin use?
Root signs in through Supabase Auth and then enters a configured 10-digit login code. The application compares keyed hashes, limits wrong code attempts to 5, and locks code entry for 15 minutes. Successful verification creates an additional Root session lasting 8 hours. This configured login code is **not an emailed OTP or a time-changing authenticator code**.

Is password reset/account recovery available?
Yes, for students and grantors. The backend requests a Supabase reset email and binds it to a hashed recovery challenge. The password is changed through Supabase, and backend completion checks the challenge and changed password before clearing the lock. The challenge has a 30-minute validity in the database implementation. Unknown or ineligible requests receive a generic response. Lost-email recovery also has private support tickets, evidence attachments and Root review; administrative recovery follows separate staff/Root controls.

How are authenticated sessions handled?
The browser uses a Supabase session with a JWT access token and refresh token. The SDK persists the session in browser storage and normally refreshes tokens; callback pages handle their own confirmation or recovery flow. API calls send Authorization: Bearer tokens. Protected backend checks resolve the user through Supabase and validate an extra database portal_verified_sessions record, account lock state and relevant revocation timestamps. Changing a browser role/ID label alone does not grant access. Root also sends an X-Root-Session token; its hash is stored on the server.

Provider-managed

Supabase issues and validates its Auth tokens and supports password-recovery sessions. Brevo delivers the application's email codes when configured. The project's extra access and verification rules remain its own responsibility.

Not Implemented / Future Improvement

No application integration for TOTP authenticator apps, passkeys or hardware security keys was found. The current browser session is not an HttpOnly-cookie session design; tokens in browser storage require strong protection against malicious scripts. Do not describe email verification as mandatory MFA for all users.

Evidence: backend/auth_service.py:115-201, 292-411; backend/access_control.py:72; backend/root_service.py:232-323; src/services/supabaseClient.js; src/services/rootService.js; [D1, D2]

2. Role Permissions and Authorization

Implemented

Which roles exist?
The main account types are **Student, Grantor/Provider, Administrator and Root Admin**. Administrator subroles are full_admin, student_reviewer, grantor_manager and reports_viewer. ISPS Coordinator describes an office responsibility; no separate coordinator account type was found. The assigned administrator role determines that person's system access.

Administrator Subrole	Permission Groups
Full admin	Students, grantors, scholarships, requirements, announcements and reports.
Student reviewer	Students and requirements; not full-admin Pending Account approval.
Grantor manager	Grantors, scholarships and announcements.
Reports viewer	Reports.

Are permissions only enforced on the screen?
No. FastAPI checks bearer authentication, the stored account/Auth-user relationship, allowed roles, permissions and ownership. Sensitive endpoints add specific full-admin or reviewer checks. The database also restricts direct access and protects selected records through grants, privileged functions, transactions and triggers.

Can a student open another student's records or files?
The inspected protected profile, application and document routes restrict students to their own records and compare the saved document owner with the authenticated student. A changed URL or browser ID is not intended to bypass those checks. Some non-private reference information is shared; student-visible roster summaries contain grantor/archive metadata, not the other scholars' full records.

Can grantors see only authorized applications?
Grantor records are scoped to their grantor ID. Related student information is limited to students linked through that grantor's applications or roster. Document access adds relationship and, for ordinary submissions, approval checks. Signed SOE access uses the separate student/authorized-admin routes inspected.

Provider-managed and Live Database Controls

PostgreSQL/Supabase supplies Row Level Security (RLS). The live read-only check found RLS enabled on all **64 public tables**, and no public-table grants to PUBLIC, anon or authenticated. Application tables include students, applications, document metadata/reviews, login state and history. Backend service-role access can bypass RLS, so server ownership checks are essential, not optional. [W3]

Not Verified / Future Improvement

This review does not prove every route is free of cross-account access errors. Maintain negative authorization tests and regularly review new endpoints and grants.

Evidence: backend/access_control.py:117-210; backend/portal_data_service.py:24-155; backend/root_service.py:941; backend/student_profile_service.py:75, 1112-1170; live metadata SELECTs on October 20, 2026

3. Sensitive Student Records

Implemented

What sensitive information is stored?
The inspected profile and document flows store Student ID, name, birthdate, email, phone, permanent address, guardian name and contact, college/course, year/section, grades/GWA, profile photos and signatures. They also store COR/ROG/identity documents, profile PDFs, scholarship applications, review results, commitments, SOE records and support/recovery evidence. Security records can contain Auth-user references, timestamps, Root IP addresses and user-agent information. Uploaded files may contain more details than the form fields listed here.

Is HTTPS/TLS used in transit?
Yes for the public website and API checked. On October 20, 2026, http://bulsuscholar.com returned a 308 redirect to HTTPS; http://api.bulsuscholar.com/health returned a 301 redirect to HTTPS. Both HTTPS requests returned 200. The website returned an HSTS header with max-age=63072000. The API health response did not return an HSTS header. This confirms those public routes, not every possible network connection.

Who can view or change records?
Students can use their own profile and upload workflows. Some fields are locked or derived from validated documents, so students cannot freely overwrite every academic/contact field. Authorized grantors see related applicants and perform permitted scholarship actions. Full admins and specialized reviewers/managers act within their assigned permissions. Root has privileged management and inspection functions. Record access is not intended to be available to all signed-in users.

Provider-managed

Does the provider encrypt stored data?
Supabase states that hosted customer data is encrypted at rest using AES-256 and in transit using TLS. This is **provider-managed encryption**, not encryption implemented by the BulsuScholar team. It does not replace role permissions or private file access. [W2]

Not Implemented / Not Verified

Does our app separately encrypt sensitive fields?
No current separate field-by-field encryption layer was found for names, addresses, grades or uploaded student documents. Application code and authorized database/service access can read these values. OTP/recovery token hashes protect verification secrets, but hashing those secrets is not encryption of student information.

Direct PostgreSQL TLS/SSL-enforcement settings and production key-rotation procedures were not verified. The example database configuration recommends sslmode=require; that recommendation alone is not proof of the live connection setting.

Evidence: backend/student_profile_service.py:50-61, 366-400, 847-859; backend/root_service.py:220-231; .env.example:58; public HTTP/HTTPS checks; [W2]

4. Uploaded Documents

Implemented

Where are documents stored? Are they public?
Document bytes are stored in **Supabase Storage**, normally in the bulsuscholar bucket. Database tables store the bucket/path, submission, revision, review and ownership information. The live bucket is **private** with a 10 MB limit. Signup files use pending-signup paths; active documents use student/cycle paths; signed SOE uses student/application paths.

How is another student's file blocked?
The main protected download/preview endpoints authenticate the caller, check their role, compare student ownership or the grantor/application relationship, and then read the saved Storage reference using backend credentials. Knowing a submission ID or path alone is not enough for those endpoints. Supabase also blocks unauthenticated access to private objects.

Are signed URLs used?
The inspected student-document and signed-SOE flows mainly use **authenticated backend content endpoints**, not temporary signed Storage URLs. The browser can turn the returned file into a temporary blob URL for the preview. A blob URL is not a signed Supabase URL. Do not claim all documents are protected through signed URLs.

Which upload checks exist?
The main backend upload routes check allowed file types, non-empty bodies, size and owner identity. COR/ROG uploads must be PDFs and pass the existing document parser and academic-policy checks. Other supported document/identity routes accept PDF, PNG, JPEG or WebP as applicable, up to 10 MB; profile photos use a 5 MB limit. Signed SOE accepts PDF/PNG/JPEG up to 10 MB. Paths are generated from the authenticated student/application. Some application file references also check HTTPS, the Supabase host and the student's path.

Provider-managed

Supabase provides private object storage and access controls. The bucket's live object-policy query returned no policies. Service-role reads bypass normal RLS rules, making the backend's authorization checks important. [W3]

Not Implemented / Future Improvement

Is malware/virus scanning implemented?
No dedicated antivirus or malware scanner integration was found. Reading COR/ROG text and checking a document's format is not a virus scan. File-type declarations alone do not prove a file is safe.

Is there a deletion/retention policy?
The code preserves submission versions and history, and has selected failed-upload cleanup and support-attachment deletion paths. No complete timed student-document retention/purge policy or scheduled cleanup of every abandoned signup file was verified. Older helpers still use direct Storage/public-URL patterns: do not treat them as private access controls or rerun the old permissive storage-policies.sql file.

Evidence: backend/student_profile_service.py:246-299, 882-940, 1112-1195; backend/security_history_service.py:105-163, 236; backend/signup_service.py:46-49; backend/scholarship_choice_service.py:169; [D3]

5. Logs and Audit Records

Implemented

Which logs actually exist?
There are several kinds, not one universal log:

Record / Table	What It Records
login_security_state	Failed-attempt count, latest success/failure, lock/unlock state and related activity timestamps. This is mainly current state, not one saved row per login attempt.
systemLogs	Workflow/system events such as selected account, document, scholarship and notification actions, with timestamps and actor information when supplied.
root_audit_logs	Root actions, actor, target, details, time, IP/user agent; includes code verification, admin management and privileged operations.
student_document_reviews / signup_account_decisions	Document preview/review actions and signup decisions, with reviewer/admin or system actor and times.
student_history_events	Student-safe application, document, roster, tracking, commitment and waitlist timeline events.
signed_soe_audit_events / public_recovery_audit_events / waitlist audit events	Dedicated events for signed-SOE changes, lost-email recovery and queue/offer actions.

Who can read them?
Root has system/audit log inspection. Full-admin access is required for the general systemLogs compatibility query. Students can read their own safe history, not staff-only notes or other students' histories. Dedicated review and workflow endpoints use their own role/permission rules.

Do logs show who changed something and when?
For supported actions, yes. Examples include account decisions, document reviews and Root actions. Root audit and selected history/audit tables have database triggers preventing ordinary updates or deletion. This is not a complete before-and-after audit of every database field, and not every history event records a human actor.

Provider-managed

Supabase and the hosting services also have operational/Auth logs. Their availability and retention depend on provider settings and plan; these were not checked in the provider dashboards.

Not Implemented / Future Improvement

How long are logs kept?
No complete time-based retention policy was found for security, document or scholarship audit logs. Root has a manual cleanup action for request metrics older than 90 days; this does **not** mean all audit logs are automatically deleted after 90 days. A full per-attempt login event trail and a consistent audit of every record change remain improvements.

Evidence: backend/supabase_ops.py:768; backend/root_service.py:220, 774, 1074; backend/portal_data_service.py:41; backend/security_history_service.py:83; [D2, D4, D5]

6. Backups and Recovery

Implemented by the Project

Does the project have a backup tool?
A manual Storage backup script exists. scripts/backup-supabase-storage.mjs downloads bucket objects to a folder outside the repository and writes a manifest with object paths, sizes and SHA-256 checksums. A September 2026 cutover note records that a manual Storage backup was made. That historical note does not prove a recent backup still exists or can be restored today.

Is database backup automatic or manual?
The repository includes deployment instructions to confirm a database restore point and separately copy Storage. **No project-maintained automated database-backup schedule was found.** No complete automated Storage-backup schedule or verified full restore tool was established by this review.

Provider-managed

Does Supabase offer automatic backups?
Yes. Supabase documents daily database backups for paid Pro, Team and Enterprise projects; availability and retention depend on the plan. Free projects should keep their own exports. This provider capability is not evidence of this project's current backup status. [W4]

Is point-in-time recovery enabled?
Not verified. PITR is a Supabase feature/add-on. The repository and the read-only metadata checks do not establish whether it is enabled or what restore points are available. Check Database > Backups and the relevant project add-on settings before making a thesis claim. [W4]

Are uploaded files included in database backups?
No. Supabase database backups include Storage metadata, not the actual uploaded file bytes. COR, ROG, IDs, signatures and SOE files require a separate Storage backup. A database-only restore cannot recreate a deleted file's contents. [W4]

Not Implemented / Not Verified

Has a restore/recovery process been tested?
No documented, independently verified end-to-end database plus Storage restore was found or run in this review. Password recovery tests and academic-cycle snapshots are not disaster-recovery tests. The actual backup plan, recent backup success, storage-copy availability, recovery-time target and acceptable data-loss target remain unverified.

Safe thesis statement: The project includes a manual Storage backup tool and deployment guidance for database backups. Scheduled backup configuration, PITR enablement and a successful full restore have not been demonstrated and must not be claimed as completed project controls.

Evidence: scripts/backup-supabase-storage.mjs; docs/deployment/SecurityCutoverStatus.md (historical September note); docs/deployment/DeploymentChecklist.md:34-40, 288; [W4]

7. Other Security Controls

Implemented

Are secrets kept in environment variables?
The backend reads service-role, email-provider, database and session/code secrets from environment variables. .env files are excluded by .gitignore and .dockerignore. The browser uses the Supabase URL and public anon key; these are not equivalent to the private service-role key. Never place private secrets in VITE_ variables, which are included in browser builds. This review does not certify all Git history or hosting environments as free of secret exposure.

Is HTTPS enforced on deployment?
Both tested public HTTP URLs redirected to HTTPS. TLS termination is supplied by the hosting layer, not FastAPI's CORS middleware. The website had HSTS; the API health response did not. Adding or verifying API HSTS and direct-database TLS enforcement remains a hardening task.

Is rate limiting implemented?
Some action-specific limits exist, not a complete global API limit. Examples are account lockout, Root code limits, email-code expiry/attempt/resend/hourly limits and a five-minute signup-confirmation resend cooldown. Signup availability checks allow 30 requests per 60 seconds per requester key, using process memory. That counter is not shared across server instances and resets on restart; it is not distributed abuse protection.

Are CORS restrictions configured?
Yes. FastAPI allows a list including the production frontend and local development origins, plus configured additional origins and an optional regex. It is not a blanket wildcard-origin policy by default. Credentials are allowed. The actual deployed extra origins/regex were not checked. CORS limits browser cross-origin behavior; it does not authenticate users or block all non-browser requests.

How is SQL injection risk reduced?
Normal workflows use structured Supabase REST/RPC calls and fixed/allow-listed tables rather than accepting ordinary users' raw SQL. Database functions use typed parameters. Root has an exceptional privileged query tool with SELECT/EXPLAIN restrictions, a read-only transaction, statement timeout and row limit. These controls reduce risk but do not prove all SQL/filter injection is impossible.

Other controls: sensitive-field redaction on compatibility responses, server ownership/path checks, disabled/pending-account restrictions, session revocation, immutable selected audit rows, database transactions and locks for critical workflows, and maintenance-mode gating.

Provider-managed / Future Improvement

Supabase supplies Auth and database security primitives; hosting supplies the observed HTTPS redirects. A verified project-wide API rate limiter, CAPTCHA integration, uniform field-level encryption and a complete documented disaster-recovery/retention program were not established. Provider WAF, network restrictions and optional Auth hardening settings remain unverified.

Evidence: backend/main.py:270-350; backend/signup_service.py:49-54, 391-399; backend/auth_service.py; backend/root_service.py:738-770; backend/student_account_review_service.py:36; .gitignore; .dockerignore

Evidence, Source References and Remaining Checks

Local Automated Verification

A focused run passed **94 tests** across test_auth_service, test_access_control, test_portal_data_service, test_security_history_service, test_student_profile_service, test_signup_confirmation, test_student_activation_service and test_root_service. These include mocked services; they are not a live malicious-user test or a proof of successful backup restoration.

Database Definitions Referenced

D1: supabase/migrations/20260918130000_automatic_roster_and_login_security.sql

D2: supabase/migrations/20260924125213_security_history_signed_soe.sql

D3: supabase/migrations/20260922170000_revoke_unsafe_direct_access.sql

D4: supabase/migrations/20260925103000_complete_student_history_ledger.sql

D5: supabase/migrations/20260912100000_secure_root_administration.sql

Document metadata/review protections are also defined in 20260923120000_student_profile_document_verification.sql. Later migrations extend these workflows. Code and migration references identify the implementation; their presence alone is not proof that every local definition is deployed.

Official Provider Sources (Checked October 20, 2026)

W1. Supabase Auth - Password Security (bcrypt hashes)

W2. Security at Supabase (hosted encryption and TLS)

W3. Supabase Storage Access Control (including service-role bypass)

W4. Supabase Database Backups (plan dependence, PITR and Storage exclusion)

What Still Needs Operational Evidence

Confirm the actual backup plan and PITR setting, record a recent database backup and separate Storage copy, and test a restore on a nonproduction project. Approve document/log retention periods and deletion responsibilities. Record the deployed backend version, complete cross-account browser/API checks, and verify production CORS, private-file routes and direct database TLS settings.

Short Word Guide

Authentication: proving who is signing in. **Authorization:** deciding which records/actions that person may access. **JWT:** a signed login token. **OTP:** a one-time code. **RLS:** database rules for row access. **TLS/HTTPS:** protection while data travels. **Encryption at rest:** protection of stored data. **Hash:** a one-way value used for comparison. **PITR:** restoring a database to a selected earlier time. **HSTS:** a browser instruction to use HTTPS. **Service role:** a powerful backend credential that must not be exposed to the browser.